MixBytes()

Fluid Money Market Oracle Security Audit Report

MARCH 17, 2026

Table of Contents

1. Introduction	**2**
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	3
1.4 Security Assessment Methodology	5
1.5 Risk Classification	7
1.6 Summary of Findings	8
2. Findings Report	**9**
2.1 Critical	9
2.2 High	9
2.3 Medium	9
M-1 No L2 Sequencer Check	9
2.4 Low	10
L-1 IMMOracle.getPrice() Missing view Modifier	10
L-2 source3 Configuration Without source2 Silently Ignored	11
L-3 Missing Staleness Check for Chainlink in Liquidate Mode	12
L-4 Silent Zero Prices in getConfiguredTokenOracles()	13
L-5 Zero Price After Negative Multiplier Application	14
L-6 _isCappedRate() Indirectly Calls Non-View Method centerPrice()	15
3. About MixBytes	**16**

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Fluid Money Market Oracle is a price oracle system developed by Instadapp for the Fluid Money Market protocol. The oracle provides token-to-USD price feeds with support for multiple price sources, different eModes, and distinct pricing for operate/liquidate modes and collateral/debt positions. The system allows configuration of up to three price sources per token configuration, with automatic fallback to eMode 0 when specific eMode pricing is unavailable. The oracle uses a multiplier system to normalize prices and implements strict validation for source addresses and freshness checks for Chainlink feeds in operate mode.

The audit examined attack vectors typical for oracle systems, including: price manipulation through source configuration, stale price feed exploitation, unauthorized configuration changes, rounding errors in price calculations, access control bypasses, and upgrade mechanism vulnerabilities. We also went through our detailed checklist, covering other aspects such as business logic, common ERC20 issues, interactions with external contracts, integer overflows, reentrancy attacks, typecast pitfalls, and other potential issues.

The scope of this audit was limited to the oracle layer contracts only. The underlying Fluid Money Market core contracts, liquidity layer, and external oracle sources (Chainlink feeds, FluidCappedRate contracts) were not included in this review.

The codebase demonstrates good architectural design with clear separation of concerns, proper access control mechanisms, and comprehensive error handling. The implementation includes appropriate validation for oracle sources and multipliers, with safeguards against stale data in critical operate mode. Several recommendations for code quality improvements and additional security considerations have been identified and documented in this report.

Key notes:

- FluidCappedRate contracts handle staleness internally: all view functions (`getExchangeRateOperate`, `getExchangeRateLiquidate`, `getExchangeRateOperateDebt`, `getExchangeRateLiquidateDebt`) check `_isHeartbeatTrigger()` and dynamically re-fetch the rate from the underlying source via `_getUpdateRates()` → `_getNewRateRaw()` when stored data is stale. See `contracts/oracle/fluidCappedRate.sol` and `contracts/oracle/fluidCappedRate.sol`.

1.3 Project Overview

Summary

Title	Description
Client	Instadapp
Category	Oracle
Project	Fluid Money Market Oracle
Type	Solidity
Platform	EVM
Timeline	13.02.2026 – 17.03.2026

Scope of Audit

File	Link
contracts/protocols/moneyMarket/oracle/errorTypes.sol	contracts/protocols/moneyMarket/oracle/errorTypes.sol
contracts/protocols/moneyMarket/oracle/error.sol	contracts/protocols/moneyMarket/oracle/error.sol
contracts/protocols/moneyMarket/oracle/proxy.sol	contracts/protocols/moneyMarket/oracle/proxy.sol
contracts/protocols/moneyMarket/oracle/events.sol	contracts/protocols/moneyMarket/oracle/events.sol
contracts/protocols/moneyMarket/oracle/variables.sol	contracts/protocols/moneyMarket/oracle/variables.sol
contracts/protocols/moneyMarket/oracle/structs.sol	contracts/protocols/moneyMarket/oracle/structs.sol
contracts/protocols/moneyMarket/oracle/interfaces.sol	contracts/protocols/moneyMarket/oracle/interfaces.sol
contracts/protocols/moneyMarket/oracle/iMMOracle.sol	contracts/protocols/moneyMarket/oracle/iMMOracle.sol

File	Link
contracts/protocols/moneyMarket/oracle/main.sol	contracts/protocols/moneyMarket/oracle/main.sol

Versions Log

Date	Commit Hash	Note
13.02.2026	5466bed98acbed39e5bfd5554bc399b527aeb670	Initial Commit
20.02.2026	d8e07d65bcf48c5441fa706d21564510adf1b2a4	Commit for re-audit
04.03.2026	8d06111c26f51f17187b8704e6e2db3ccb3cf51c	Commit for re-audit
05.03.2026	698e6a9149bd5dc0185340370f3566093a82444f	Commit for re-audit
17.03.2026	6b0eb87835e2b4e5bd03ed9ef21e527e67318729	Commit for re-audit

Mainnet Deployments

Contract name	Contract deployed on mainnet	Blockchain
FluidMoneyMarketOracle	0xA5B62E37Ef85Cf83854828C5942BCbaF3300eB61	Ethereum
FluidMoneyMarketOracleProxy	0x722065CBf94aCE5d4966a03082D933efb8055233	Ethereum

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	**Project Architecture Review:** • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: · Cross-check findings among auditors · Discuss identified issues · Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	**Bug Fixing & Re-Audit:** · The client addresses the identified issues and provides feedback · Auditors verify the fixes and update their statuses with supporting evidence · A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	**Final Code Verification & Public Audit Report:** · Verify the final code version against recommendations and their statuses · Check deployed contracts for correct initialization parameters · Confirm that the deployed code matches the audited version · Issue a public audit report, published on our official GitHub repository · Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likehood: High	Critical	High	Medium
Likehood: Medium	High	Medium	Low
Likehood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	1
Low	6

Findings Statuses

ID	Finding	Severity	Status
M-1	No L2 Sequencer Check	Medium	Acknowledged
L-1	IMMOracle.getPrice() Missing view Modifier	Low	Acknowledged
L-2	source3 Configuration Without source2 Silently Ignored	Low	Fixed
L-3	Missing Staleness Check for Chainlink in Liquidate Mode	Low	Fixed
L-4	Silent Zero Prices in getConfiguredTokenOracles()	Low	Fixed
L-5	Zero Price After Negative Multiplier Application	Low	Fixed
L-6	_isCappedRate() Indirectly Calls Non-View Method centerPrice()	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	No L2 Sequencer Check		
Severity	Medium	**Status**	Acknowledged

Description

When the L2 sequencer is down, users can still submit transactions via the L1 inbox (force-include). The oracle does not use Chainlink's L2 Sequencer Uptime Feed, so it keeps serving prices. That creates an inequality: only users who can use the L1 path can act, and they do so against stale prices (Chainlink on L2 is not updated while the sequencer is down). They can operate at non-market prices.

- contracts/protocols/moneyMarket/oracle/main.sol

Recommendation

We recommend following chainlink's recommendation for L2 sequencer uptime feed at https://docs.chain.link/data-feeds/l2-sequencer-feeds

Client's Commentary:

The first version is intended for mainnet only. On L2s, we plan to include the same sequencer check that already exists in our earlier oracle implementations: contracts/oracle/fluidOracleL2.sol

2.4 Low

L-1	IMMOracle.getPrice() Missing view Modifier		
Severity	Low	**Status**	Acknowledged

Description

The IMMOracle interface declares getPrice() without the view modifier, while the implementation in FluidMoneyMarketOracle correctly uses view. This mismatch prevents other contracts from calling getPrice() from their own view functions, as Solidity's compiler will not allow calling a non-view function from a view context.

Code References:

- contracts/protocols/moneyMarket/oracle/iMMOracle.sol

Recommendation

We recommend adding the view modifier to the interface function signature.

Client's Commentary:

This behavior is intentional. We want protocols to integrate with the oracle in a way that assumes there might be a write operation performed by the oracle itself. This is because we have identified certain cases where we may want to trigger a price update (or similar action) directly from within the oracle, and we want to preserve maximum flexibility on the oracle side for potential future changes in this direction. Related: contracts/oracle/fluidCappedRate.sol

L-2	source3 Configuration Without source2 Silently Ignored		
Severity	Low	**Status**	Fixed in d8e07d65

Description

The _parseAndValidateOracleConfig() function allows setting sourceCfg1 and sourceCfg3 while skipping sourceCfg2. The configuration is successfully saved, but source3 will never be used in getPrice because the function returns early when sourceType2 == 0. This creates a misleading configuration where governance believes they have configured three sources, but only the first one is actually used.

- contracts/protocols/moneyMarket/oracle/main.sol

Recommendation

We recommend either documenting this behavior or adding validation to ensure that if sourceCfg3 is set, sourceCfg2 must also be set.

L-3	Missing Staleness Check for Chainlink in Liquidate Mode		
Severity	Low	**Status**	Fixed in d8e07d65

Description

The _readChainlinkSource() function only checks for staleness when isOperate_ is true. In liquidate mode, no staleness check is performed at all, allowing liquidations to proceed with potentially very stale prices. While this may be intentional design (to avoid blocking liquidations when oracle feeds have issues), it creates a risk of liquidations occurring at significantly outdated prices.

For example, a user who borrowed ETH against wstETH collateral may maintain healthy position ratios during a correlated market downturn (both tokens falling together), but if the ETH oracle feed becomes stale and reports an outdated higher price while wstETH collateral keeps going down, reflecting current market conditions, the system will calculate health using inflated debt value against deflated collateral, potentially triggering an unjustified liquidation.

- contracts/protocols/moneyMarket/oracle/main.sol

Recommendation

We recommend adding a more lenient staleness check for liquidate mode (e.g., 7 days) to prevent using completely dead feeds while still allowing liquidations to proceed even with slightly stale data. Alternatively, document this as an intentional design decision.

L-4	Silent Zero Prices in getConfiguredTokenOracles()		
Severity	Low	**Status**	Fixed in d8e07d65

Description

In the getConfiguredTokenOracles() view function, if readSource() fails for rate2 or rate3 (caught by try/catch), the rate remains zero. The final price calculation then multiplies by zero, resulting in a zero price being returned as valid without any error indication. This can mislead off-chain consumers who may interpret zero prices as valid configurations.

- contracts/protocols/moneyMarket/oracle/main.sol

Recommendation

We recommend documenting this behavior clearly for off-chain consumers.

L-5	Zero Price After Negative Multiplier Application		
Severity	Low	**Status**	Fixed in d8e07d65

Description

In the _readSource() function, the zero check is performed **before** applying the multiplier. When a source returns a small value (e.g., 1000) and a negative multiplier is applied (e.g., -4), the value can become zero after division: 1000 / 10000 = 0. This zero value is returned as a valid price without any error, which may lead to incorrect valuations.

- contracts/protocols/moneyMarket/oracle/main.sol

Recommendation

We recommend either documenting this behavior or moving the zero check after the multiplier application to ensure that the function cannot return zero.

L-6	_isCappedRate() Indirectly Calls Non-View Method centerPrice()		
Severity	Low	**Status**	Acknowledged

Description

During the configuration update process, the _isCappedRate() helper indirectly calls the centerPrice() method on the IFluidCappedRate contract. The centerPrice() function is not declared as view, so invoking it from _isCappedRate() (which is also non-view) may lead to unexpected state mutations during configuration updates (e.g., if centerPrice() is implemented with side effects). This slightly increases the risk surface for governance or multisig operations that are expected to be pure configuration changes.

- contracts/protocols/moneyMarket/oracle/main.sol
- contracts/protocols/moneyMarket/oracle/main.sol

Recommendation

We recommend either documenting this behavior or changing centerPrice() in the IFluidCappedRate implementation to be view if it is logically read-only, or refactoring _isCappedRate() to rely only on strictly view-compatible checks (e.g., using read-only getters) to avoid any possibility of unintended state changes during configuration validation.

Client's Commentary:

centerPrice() must stay non-view, this is intended (see L-1 comment)

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**,**Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information

https://mixbytes.io/

https://github.com/mixbytes/audits_public

https://x.com/MixBytes

hello@mixbytes.io